数据分析知识点复习

一、 语法

1. PEP8 编码规范与对象模型

- **代码注释**：单行注释使用 `#`；多行注释使用三对双引号 `"""` 或单引号 `'''` 包裹。
- **命名规范**：
  - **全局变量**：全大写加下划线（如 `MAX_VALUE`）。
  - **私有变量**：小写字母加单前导下划线（如 `_private_var`）。
  - **内置/魔术变量**：双前导和双后置下划线（如 `__init__`）。
  - **类名**：首字母大写/驼峰式（如 `ClassName`）。
- **空格规范**：
  - 赋值/比较/逻辑运算符（`=`, `==`, `and` 等）前后**加**空格。
  - 高优先级运算符（`*`, `/`, `**`）前后**不加**空格。
- **对象三要素**：
  - **身份 (Identity)**：内存地址标识，用 `id()` 获取，产生后不可改变。
  - **类型 (Type)**：数据分类，用 `type()` 获取。
  - **值 (Value)**：存储的具体数据。
- **连环赋值**：`a = b = c = 1` 会将三个变量绑定到同一内存地址。

2. 核心数据类型

- **整型 (int)**：支持无限精度，大小仅受限于运行环境内存。十六进制以 `0x` 开头，八进制以 `0o` 开头。空参 `int()` 默认返回 `0`。
- **浮点型 (float)**：遵循 IEEE754 双精度标准，独占 8 个字节内存。
- **复数 (complex)**：由实部和虚部（以 `j` 或 `J` 结尾）组成。**注意**：复数只能比较是否相等（`==`），绝对不能比较大小（`>` 或 `<`）。
- **布尔型 (bool)**：
  - 是整型的子类（`False` 等效于 `0`，`True` 等效于 `1`）。
  - **隐式假值**：数值 `0`、`None`、空字符串 `""`、空列表 `[]`、空元组 `()`。
  - **陷阱**：`bool("False")` 结果为 `True`（因为是非空字符串）；`bool([0])` 结果为 `True`（因为是非空列表）。

3. 运算符与优先级

- **算术运算符**：
  - `//`：取整除（向负无穷取整数商）。
  - `%`：取模（求余数）。
  - `**`：幂运算。具有**右结合性**（如 `2 ** 3 ** 2` 等价于 `2 ** 9`），且优先级高于单目负号（如 `-3 ** 2` 等价于 `-(3 ** 2)`，结果为 `-9`）。
- **逻辑运算符**：
  - 优先级自高到低：`not` > `and` > `or`。
  - **短路机制**：如 `False and (10/0)` 会直接返回 `False`，不会引发除零错误。
- **链式比较**：`1 < 2 == 2` 是合法的，等效于 `(1 < 2) and (2 == 2)`。

二、 流程控制

- **代码块边界**：严格依赖**行首缩进**来划分代码块，不使用大括号。
- **条件分支 (`if-elif-else`)**：
  - 只要触发匹配项就会跳出整个结构（排他性）。
  - 若直接并列多个独立的 `if`，则每个条件都会被强制判定一次。
- **循环遍历 (`for` / `while`)**：
  - `range(start, stop, step)`：遵循**左闭右开**原则，永远取不到 `stop` 的边界值。
- **循环控制字**：
  - `break`：立刻终结并跳出**当前这一层**循环。
  - `continue`：中断**本次**迭代的剩余代码，直接进入下一轮条件判定。
- **循环 `else` 分支**：
  - **触发条件**：只有当循环**正常执行完毕**时才会执行。
  - **失效条件**：一旦循环体内部触发了 `break`，`else` 代码块将被直接抛弃，绝对不会执行。

三、 组合数据类型

- 常见的不可变对象
  - **数字类型**：`int`，`float`，`bool`，`complex`
  - **字符串**：`str`
  - **元组**：`tuple`
  - **冻结集合**：`frozenset`
  - **字节串**：`bytes`
- 常见的可变对象
  - **列表**：`list`
  - **字典**：`dict`
  - **集合**：`set`
  - **字节数组**：`bytearray`
  - 用户自定义的类实例

Python 提供了四种主要的组合数据类型，按**是否可变**和**是否有序**区分：

1. 列表 (List) - `[]`

- **特点**：有序、可变、可变长、允许重复、支持异构嵌套。
- **访问**：通过索引访问（如 `list[0]`，`list[-1]`），支持切片（`list[start:end:step]`）。
- **常用方法**：
  - 增加：`append(x)`（末尾追加）、`insert(i, x)`（定点插入）、`extend(seq)`（合并扩展）。
  - 删除：`del list[i]`、`remove(x)`（删除首个匹配值）、`pop(i)`（弹出元素，默认末尾）、`clear()`。
  - 其他：`reverse()`（反转）、`sort(key=None, reverse=False)`（排序）、`max()`（最大值）、`min()`（最小值）、`copy()`（复制列表）、`count(x)`（统计该元素在列表中出现的次数）。

2. 元组 (Tuple) - `()`

- **特点**：有序、**不可变**（防弹衣版列表）、可变长、允许重复、支持异构嵌套。
- **定义陷阱**：单元素元组必须加逗号，即 `(1,)`，否则括号会被视为算术运算符。
- **操作**：支持索引和切片，不支持修改和删除内部元素，但可用 `del tuple` 删除整个元组对象。

3. 字典 (Dictionary) - `{key: value}`

- **特点**：无序、可变、**键唯一且不可变**（如数字、字符串、元组）。
- **操作**：
  - 读取/修改：`dict[key]` 或 `dict.get(key)`。
  - 删除：`del dict[key]`、`pop(key)`、`clear()`。
  - 视图：`keys()`、`values()`、`items()`。

4. 集合 (Set) - `{}` 或 `set()`

- **特点**：无序、可变、**元素唯一**（自动去重）。
- **定义陷阱**：创建空集合必须使用 `set()`，`{}` 默认是空字典。
- **运算**：支持数学集合运算。交集 `&`、并集 `|`、差集 `-`、对称差集 `^`。
- **操作**：`add(x)`（添加）、`update(seq)`（合并）、`remove(x)`（删除，不存在报错）、`discard(x)`（删除，不存在不报错）。

四、 函数

1. 函数基础

- **定义**：使用 `def` 关键字，以冒号 `:` 结尾，依靠**缩进**划分函数体。
- **返回值**：使用 `return`。不写 `return` 时隐式返回 `None`；支持返回多个值（底层自动打包为元组）。

2. 变量作用域

- **局部变量**：函数内部定义，仅在函数内有效。
- **全局变量**：函数外部定义。函数内可读取，若要在函数内修改全局变量，必须使用 `global` 关键字声明。

3. 五种参数传递方式

- **位置参数**
  - **特点**：严格按照函数定义时的参数**顺序**依次传入，形参与实参必须一一对应。
- **关键字参数**
  - **特点**：在调用时通过 `参数名=值` 的形式显式指定（如 `func(name='Alice')`）。
- **默认值参数**
  - **特点**：在定义函数时直接为形参赋予初始值。调用时若不传入该参数，则自动使用默认值。
  - **注意**：定义时**必须**放置在常规位置参数的后面；强烈建议使用**不可变对象**（如数字、字符串、元组）作为默认值，以防多次调用时产生数据污染。
- **可变长位置参数 (*args)**
  - **特点**：用于收集数量未知、多余的位置参数，系统会在函数内部自动将其打包为一个**元组**。
  - **传参方式**：可以直接传入多个用逗号隔开的值（如 `func(1, 2, 3)`）；也可以传入一个已有的列表或元组，并在其变量名前加上 `*` 进行解包操作（如 `func(*my_list)`）。
- **可变长关键字参数 (**kwargs)**

- **特点**：用于收集数量未知、多余的关键字参数，系统会在函数内部自动将其打包为一个**字典。
- **传参方式**：可以连续以 `key=value` 的形式传入（如 `func(a=1, b=2)`）；也可以传入一个已有的字典，并在其变量名前加上 `**` 进行解包操作（如 `func(**my_dict)`）。

4. 高阶概念

- **闭包**：内部函数引用了外部嵌套函数（非全局）的局部变量。
- **递归**：函数调用自身。必须有**退出条件**，Python 默认最大递归深度为 1000 层（可通过 `sys.setrecursionlimit()` 修改）。

五、 类与对象

1. 面向对象基础 (OOP)

- **核心概念**：
  - **面向对象编程 (OOP)**：以“对象”为基本单元，用模型反映现实世界事物特征与规律的抽象思维。
  - **类 (Class)**：创建对象的图纸/代码段。命名遵循 **PEP8 首字母大写 (CapWords)** 规范。
  - **对象/实例 (Object/Instance)**：根据“类”这张图纸造出来的具体实体。
- **核心关键字与方法**：
  - `class`：用于定义类的关键字。
  - `self`：类的实例方法中**必须存在的第一个默认参数**，代表对象自身（类似于 C++/Java 的 `this`）。
  - `__init__(self, ...)`：**构造/初始化方法**。在对象分配内存后自动调用，用于绑定和初始化实例属性。
  - `__del__(self)`：**析构方法**。在对象生命周期结束、被内存释放回收时自动调用。

2. 面向对象三大特性

1. 封装 (Encapsulation)

- **概念**：将属性和方法集中到类中，对外部隐藏内部细节，提升安全性。
- **伪私有变量 (Name Mangling)**：
  - **定义**：在变量或方法名前加**双下划线**（如 `__assets`）。
  - **机制**：Python 底层会自动将其重命名为 `_类名__变量名`（如 `_Company__assets`）。
  - **考点**：Python 中**没有真正的绝对私有**，外部依然可以通过 `实例._类名__变量名` 强制访问。未加下划线的属性默认都是公有的 (public)。

2. 继承 (Inheritance)

- **概念**：子类自动拥有父类的所有属性和方法，极大**降低代码冗余，提高复用性**。
- **单继承**：`class Child(Parent):`
- **多重继承**：`class Child(Base1, Base2):`。支持同时继承多个父类，但易导致代码混乱，需谨慎使用。
- **查找顺序 (MRO)**：在多重继承中，寻找同名方法的底层算法顺序被称为 **MRO (Method Resolution Order)**。
- `super()` **函数**：
  - **作用**：在子类重写方法时，用于安全地**调用父类的同名方法**。
  - **优势**：自动绑定 `self`，自动处理多重继承的 MRO 寻址，无需硬编码父类名称（如 `super().func()`）。

3. 多态 (Polymorphism)

- **概念**：同一种指令（方法调用），作用于不同的对象实例时，表现出不同的行为状态。

- **实现**：通常通过子类**重写 (Override)** 父类的同名方法来实现。就近原则，子类重写后，默认执行子类的方法。

六、 异常

- **异常 vs 错误**：语法错误 (SyntaxError) 在编译期拦截；异常 (Exception) 是程序在**动态运行期间**遇到的特殊危机事件。

1. 异常处理 (`try-except-else-finally`)

- `try:`
  - 存放可能发生异常的“危险代码”。一旦某行报错，该块**剩余代码立刻停止执行**，跳转至 `except`。
- `except Exception as e:`
  - 捕获并处理异常。`e` 是被捕获的**异常对象实例**，包含错误细节。
  - **坑点**：捕获多种异常必须用**元组**打包：`except (ValueError, TypeError):`。极不推荐使用不带类型的“裸 `except:`”。
- `else:`
  - **触发条件**：`try` 块内代码**完全正常执行完毕，未抛出任何异常**时触发。
- `finally:`
  - **触发条件**：**无论是否发生异常，最后必定会执行。**
  - **应用场景**：释放资源、关闭文件句柄、断开数据库连接等硬性清理工作。

2. 自定义异常与主动抛出

- **自定义异常**：必须继承自 `Exception` 基类（或其子类）。通常内部只需写 `pass` 占位即可，核心意义是定制一个符合业务逻辑的“错误标签”（如 `InsufficientFundsError`）。
- `raise` **关键字**：用于在代码中**主动触发/抛出**一个异常对象（如 `raise ValueError("输入不合法")`）。

3. 上下文管理器 (`with` 语句)

- **作用**：简化资源管理代码（如 `with open('file.txt') as f:`）。
- **优势**：即使在文件处理过程中发生异常，也会在离开 `with` 代码块时**自动、安全地关闭文件**，相当于底层替你写好了 `finally: f.close()`。

基本输入输出：

- `print()`：输出指定内容。默认输出后会换行，添加参数 `end=''` 可取消换行。
- `input()`：接收用户键盘输入，默认获取到的数据类型一律为字符串 (`str`)。

七、 文件

1. 文件的打开与关闭

- **基础语法**：`file_obj = open(file_name, mode, buffering)`
- **核心访问模式 (Mode)**：
  - `r`：只读（默认模式）。
  - `w`：写入。若文件存在则**覆盖**，不存在则创建。
  - `a`：追加。内容写在文件**末尾**，不破坏原有数据。
- **缓冲模式 (Buffering)**：
  - `0`：无缓冲（实时写入）。

    - `1`：行缓冲（遇到换行符 `\n` 写入）。
    - `>1` (如 4096)：全缓冲（默认，写满指定字节数后写入）。
- **关闭文件**：`f.close()`。必须执行以释放资源和清空缓冲区。

2. 文件的读与写

- **读操作**：
    - `read(size)`：读取指定字节。不传参则**一次性读取整个文件**，返回大字符串。
    - `readline()`：读取**单行**内容（包含末尾的 `\n`）。
    - `readlines()`：读取**所有行**，返回一个**列表 (List)**，每行是列表的一个元素。
- **写操作**：
    - `write(string)`：将字符串写入文件。**注意**：不会自动添加换行符，需手动加 `\n`。

3. 系统与目录操作 (`os` 模块)

操作前必须 `import os`：

- **文件级**：
    - `os.rename(old, new)`：重命名文件。
    - `os.remove(file)`：删除文件。
- **目录级**：
    - `os.mkdir(dir)`：创建新目录。
    - `os.rmdir(dir)`：删除空目录。
    - `os.getcwd()`：获取当前工作目录 (Get Current Working Directory)。
    - `os.chdir(dir)`：改变当前工作目录 (Change Directory)。

八、 字符串

1. 基础与切片

- **合并与重复**：`'A' 'B'` 自动合并为 `'AB'`；`'Jack' * 3` 变为 `'JackJackJack'`。
- **索引**：从左到右 `0` 开始；从右到左 `-1` 开始（`-1` 代表倒数第一个）。
- **切片** `[start:end:step]`：
    - **口诀**："包左不包右"。例如 `s[1:4]` 截取索引 1, 2, 3，不包含 4。
    - `step`：步长，默认为 1。例如 `s[1:7:2]` 代表隔一个取一个。

2. 格式化输出

- `%` **表达式**：
    - `%s` (字符串), `%d` (十进制整数), `%f` (浮点数)。
    - 考点：保留两位小数使用 `%.2f`。
- `format()` **函数**：
    - `'I am {0}, {1} years old'.format('Jack', 18)`。

3. 常用核心方法

- **清理**：`strip()` 默认去除首尾空格和换行符。
- **统计**：`count(sub)` 统计子串出现的次数。
- **大小写**：`capitalize()` (仅首字母大写)，`lower()` (全小写)，`upper()` (全大写)。
- **查找与判断**：
  - `find(sub)`：找不到返回 `-1`。
  - `index(sub)`：找不到直接 **报错 (ValueError)**。
  - `startswith(sub)` / `endswith(sub)`：判断是否以指定子串开头/结尾。
- **内容类型判断**：
  - `isalnum()`：是否仅由字母**或**数字组成。
  - `isalpha()`：是否纯字母。
  - `isdigit()`：是否纯数字。
- **分割与替换**：
  - `split(sep)`：按分隔符拆分为**列表**。
  - `partition(sep)`：拆分为前、中、后三部分，返回**元组**。
  - `replace(old, new, count)`：默认替换所有出现的旧子串。

九、 正则表达式

1. 核心正则符号

- **匹配字符**：
  - `\d`：数字 (0-9)。
  - `\w`：单词字符（字母、数字、下划线）。
  - `\s`：空白字符。
  - 大写字母 (`\D`, `\W`, `\S`) 代表反义（非数字、非单词、非空白）。
  - `[]`：匹配括号内的任意**一个**字符（如 `[abc]`）。
- **数量匹配 (量词)**：
  - `*`：0次或无限次（可有可无，多多益善）。
  - `+`：1次或无限次（至少有一次）。
  - `?`：0次或1次（最多只能出现一次）。
  - `{m}`：精确出现 m 次。

2. `re` 模块核心方法

- `re.match()` **vs** `re.search()` (必考区别)：
  - `match()`：死板。只从字符串**最开头**匹配，开头不符合直接返回 `None`。
  - `search()`：灵活。在全文中**全局扫描**，找到第一个符合的就返回，全篇没有才返回 `None`。
- **获取结果**：成功匹配后，调用返回对象的 `group()` 方法提取字符串。
- **模式编译**：`re.compile(pattern)` 用于预编译正则图纸，提升重复使用的执行效率。
- **实战扩展**：`re.findall()` 用于找出全文所有匹配项并返回一个**列表**。

十、 Pandas

1. 数据读取与预览

- **读取文件**：`pd.read_csv("文件路径或网络URL")`，专用于读取逗号分隔值文件。
- **数据结构**：`DataFrame` 本质上是由共享索引的多个 `Series` 组合而成的二维数据表。
- **快速预览**：
  - `df.head()`：默认查看前 5 行。
  - `df.tail()`：默认查看末尾 5 行。
  - `df.describe()`：快速输出连续型数值列的统计描述（最大/小值、均值、标准差、分位数）。

2. 数据清洗

- **目的**：处理缺失值、异常值和重复数据，为后续分析或机器学习提供干净的数据基座（模型通常无法直接处理包含 `NaN` 的数据）。
- **核心方法**：
  - `dropna()`：删除包含缺失值的行或列。
  - `fillna()`：使用特定值（如均值、中位数）填充缺失值。
  - `drop_duplicates()`：删除完全重复的数据行。

十一、 数据可视化

1. Matplotlib (`pyplot`)

- **核心概念**：
  - **Figure**：最底层的画布对象（`plt.figure(figsize=(宽,高))` 用于初始化全局尺寸）。
  - **子图**：`plt.subplot()` 可以在同一张大画布中切分多个独立坐标系区域。
- **常用图表**：
  - 折线图：`plt.plot()`（默认将二维数组的各列视为独立的线）。
  - 散点图：`plt.scatter()`。
  - 柱状图：`plt.bar()`。
- **图表定制与输出**：
  - `plt.title()` (标题), `plt.xlabel()` (X轴标签), `plt.legend()` (图例，基于图元的 `label` 参数生成)。
  - 展示与保存：`plt.show()` 弹出图形窗口，`plt.savefig('name.png')` 将图像导出到本地。

十二、 NumPy 基础

1. 数组核心属性

- `ndim`：维度数/秩（如二维数组返回 2）。
- `shape`：形状特征，返回**元组**（如 `(3, 4)` 代表 3行4列）。
- `size`：数组内元素的总个数（形状各维度乘积）。
- `itemsize`：每个元素在内存中占用的**字节数**（例如 `int32` 返回 4字节）。

2. 生成与运算

- **生成**：`np.zeros()` (全零数组), `np.arange(start,` stop, step) (等差序列)。**注意**：多维数组的各行/列长度必须严格一致，否则会退化为一维对象数组。
- **科学计算 (ufunc)**：`np.sin()`, `np.mean()` (算术平均)。底层由 C 语言实现并在连续内存上操作，性能远超原生 Python 列表。
- **矩阵运算**：`np.dot()` 代表线性代数中的矩阵内积乘法；而 `*` 代表逐元素相乘。
- **广播机制 (Broadcasting)**：自动对较小形状的数组进行补齐拉伸，以实现不同形状数组的无缝算术运算。
- **文件存储**：`np.savez()` 将多个数组打包保存为非压缩的 `.npz` 格式；加载后可通过字典键名（如 `r['arr_0']`）提取。

十三、 NumPy 轴操作

1. Axis 轴向解析

- `axis=0`：沿着行方向（纵向扫过）。
- `axis=1`：沿着列方向（横向扫过）。
- `axis=-1`：永远动态指向最内层的最后一个维度。

2. 聚合与状态保留

- **聚合函数**：`sum()`, `mean()`, `max()` 等会导致被操作的维度坍缩消失。
- **保持维度**：加入 `keepdims=True` 参数，坍缩的维度将被保留为 `1`（如变为 `(n, 1)`），便于后续广播计算而不抛出维度错误。
- **累加累积**：`np.cumsum()` (前缀和) 与 `np.cumprod()` (前缀乘积)，操作后维度不坍缩。

3. 带 "arg" 前缀的索引函数

- **核心特性**：不返回极值本身，而是返回**极值所在的索引/坐标**。
- **常用操作**：
  - `np.argmax(arr, axis=1)`：找寻每一行中最大元素所在的列标号。
  - `np.argsort()`：返回排序操作后的索引数组（指导牌），常用于不打散行的间接排序。**警告**：直接对矩阵使用 `np.sort(arr,` axis=0) 会破坏行内数据的绑定关系。

4. 堆叠与拼接

- **在已有轴上拼接 (`concatenate`)**：不增加维度。必须确保非拼接轴的维度严格相等。简写有 `np.hstack()` (水平/列), `np.vstack()` (垂直/行)。
- **创建新轴堆叠 (`stack`)**：结果的维度 `ndim` 一定比原数组多 1 阶。

5. 形状重塑与操作

- **维度调整**：
  - `reshape()`：安全改变维度结构（如利用 `-1` 自动计算长度）。
  - `expand_dims(arr, axis)`：在指定位置插入一个大小为 1 的气泡维度（升维）。
  - `squeeze()`：挤破/消除长度仅为 1 的冗余维度（降维）。
  - `swapaxes(arr, ax1, ax2)`：精准互换两个轴的位置。
  - `transpose()` 或 `.T`：重排矩阵维度视角。
- **展平操作**：`flatten()` 将任意维度的数组碾平为一维（通常采用行优先读取）。

6. 条件逻辑与高级筛选

- `np.where(condition, x, y)`：矢量化的三目运算。若省略 x 和 y，则充当寻址仪，返回符合条件的索引元组。
- **布尔逻辑判定**：
  - `np.any(axis)`：指定轴上存在**至少一个** `True` 即返回 `True`。
  - `np.all(axis)`：指定轴上**全员满足** `True` 才返回 `True`。
- **多条件组合**：在 NumPy 的索引或 `where` 语句中，必须使用按位逻辑运算符 `&` (与)、`|` (或)，并用 `()` 将各个独立条件包裹。
- **边界截断**：`np.clip(arr, min, max)`：将超出范围的值强行修剪/拔高到指定的边界值范围内。
- **唯一值**：`np.unique()` 返回去重并自动升序排列的结果。
- **浮点数比较**：避免使用 `==`，推荐使用 `np.isclose()` 或 `np.allclose()` 以容忍微小的浮点误差